

Member	Belongs to	Access from main()
boolean defaultValue;	Object (instance)	☐ Need an object
static boolean defaultValue;	Class	☐ Directly accessible

Character Escape Sequences

```

public class EscapeExample
{
    public static void main(String args[])
    {
        String str = new String ("My name is \'abcd\'");
        System.out.println(str);

        str = "My Lab is \"JAVA LAB\"";
        System.out.println(str);

        str = "My work files are in D:\\Work Projects\\java";
        System.out.println(str);

        str = "First Line \nSecond Line";
        System.out.println(str);

        str = "Ravindra\b\b" + "College";
        System.out.println(str);

        str = "Ravindra \rCollege";
        System.out.println(str);

        str = "Ravindra \fCollege";
        System.out.println(str);
    }
}

```

```

C:\Users\RCEW LAB 1>java EscapeExample
My name is 'abcd'
My Lab is "JAVA LAB"
My work files are in D:\Work Projects\java
First Line
Second Line
RavindCollege
Collegea
Ravindra  College
C:\Users\RCEW LAB 1>

```

Variables

The variable is the basic unit of storage in a Java program.

- A variable is defined by the combination of an identifier, a type, and an optional initializer.

- In addition, all variables have a scope, which defines their visibility, and a lifetime

```
int a, b, c;           // declares three ints, a, b, and c.
int d = 3, e, f = 5;   // declares three more ints, initializing
                      // d and f.
byte z = 22;           // initializes z.
double pi = 3.14159;   // declares an approximation of pi.
char x = 'x';          // the variable x has the value 'x'.
```

```
// Demonstrate dynamic initialization.
class DynInit {
    public static void main(String args[]) {
        double a = 3.0, b = 4.0;
        // c is dynamically initialized
        double c = Math.sqrt(a * a + b * b);
        System.out.println("Hypotenuse is " + c);
    }
}
```

The Scope and Lifetime of Variables

```
// Demonstrate block scope.
class Scope {
    public static void main(String args[]) {
        int x; // known to all code within main
        x = 10;
        if(x == 10) { // start new scope
            int y = 20; // known only to this block
            // x and y both known here.
            System.out.println("x and y: " + x + " " + y);
            x = y * 2;
        }
        // y = 100; // Error! y not known here
        // x is still known here.
        System.out.println("x is " + x);
    }
}
```

Scanner

- `public String nextLine()` :- Moves the scanner to the next line and returns the skipped input.
- `public String next()` :- Returns the token before delimiter
- `public byte nextByte()` :- Scans next token as byte value
- `public short nextShort()` :- Scans next token as short value
- `public int nextInt()` :- Scans next token as integer value
- `public long nextLong()` :- Scans next token as long value
- `public float nextFloat()` :- Scans next token as float value
- `public double nextDouble()` :- Scans next token as double value
- `void close` :- Scanner is closed

next(): Read One word stops at Space, tab, or newline

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your full name: ");
        String name = sc.next();

        System.out.println("Name = " + name);

        sc.close();
    }
}
```

Input: Ashok Rajan

Output: Name = Ashok

nextLine(): Read the entire line

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter your full name: ");
String a = sc.nextLine();
System.out.println(a);
```

Input: Ashok Rajan R
Output: Ashok Rajan R

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Read byte
        System.out.print("Enter a byte value: ");
        byte b = sc.nextByte();

        // Read short
        System.out.print("Enter a short value: ");
        short s = sc.nextShort();

        // Read int
        System.out.print("Enter an int value: ");
        int i = sc.nextInt();

        // Read long
        System.out.print("Enter a long value: ");
        long l = sc.nextLong();

        // Read float
        System.out.print("Enter a float value: ");
        float f = sc.nextFloat();

        // Read double
        System.out.print("Enter a double value: ");
        double d = sc.nextDouble();

        System.out.println("\n---- Values Entered ----");
        System.out.println("Byte   : " + b);
        System.out.println("Short : " + s);
        System.out.println("Int    : " + i);
        System.out.println("Long   : " + l);
        System.out.println("Float  : " + f);
        System.out.println("Double : " + d);

        sc.close();
    }
}
```

```
}  
}  
  
Enter a byte value: 100  
Enter a short value: 20000  
Enter an int value: 500000  
Enter a long value: 9876543210  
Enter a float value: 12.5  
Enter a double value: 45.6789  
  
----- Values Entered -----  
Byte : 100  
Short : 20000  
Int : 500000  
Long : 9876543210  
Float : 12.5  
Double : 45.6789
```

Non-Primitive (Reference) Data Types

Non-primitive data types are derived from primitive types or created by programmers.

One-Dimensional Arrays

```
public class Main {  
    public static void main(String[] args) {  
  
        int[] arr = {10, 20, 30, 40, 50};  
  
        System.out.println("Array Elements:");  
  
        for (int i = 0; i < arr.length; i++) {  
            System.out.println(arr[i]);  
        }  
    }  
}
```